MDL-001 — Mosaic Design Language Vision

External publication draft

MDL-001 — Mosaic Design Language Vision

> *Mosaic exists to remove the friction between people and the entertainment they love.*

Purpose

MDL-001 is the constitutional document of the Mosaic Design Language (MDL).

Unlike traditional design specifications, this document does **not** define colours, typography, layouts or components.

Instead it defines **why** Mosaic exists.

Every future design and engineering decision should be traceable back to the philosophy established here.

If another MDL or MDS specification appears to conflict with MDL-001, that specification should be reconsidered before this document is modified.

Scope

This specification defines:

- Product vision
- Product beliefs
- Design philosophy
- Long-term objectives
- Contributor mindset
- Design governance

This specification intentionally does **not** define:

- Colour systems
- Typography
- Motion
- Components
- Design tokens
- Layout rules
- Engineering implementation

Those concerns are defined by later MDL and MDS specifications.

Specification Structure

- 00 Document Control
- 01 Background & Problem Statement

- 02 Vision
- 03 Product Beliefs
- 04 Goals
- 05 Non-Goals
- 06 Design Philosophy
- 07 Governance
- 08 Architectural Decision Records
- 09 Contributor Guidance
- 10 Design Review Checklist
- 11 Future Considerations
- Glossary
- References

Dependencies

This specification has no dependencies.

All remaining MDL specifications depend on MDL-001.

```
[mermaid]
flowchart TD

MDL001["MDL-001 Vision"]

MDL001 --> MDL002["MDL-002 Principles"]
MDL001 --> MDL003["MDL-003 Mental Model"]
MDL001 --> MDL004["MDL-004 Interaction Model"]
MDL001 --> MDL005["MDL-005 Composition Model"]

MDL001 --> MDS001["MDS-001 Design Token Architecture"]
MDL001 --> MDS002["MDS-002 Material System"]
MDL001 --> MDS003["MDS-003 Composition Engine"]
```

Repository Convention

Each chapter exists as an individual Markdown document.

This intentionally mirrors software architecture.

Small documents:

- review more easily
- version more cleanly
- minimise merge conflicts
- encourage focused discussion

- can be assembled into PDF, mdBook or documentation websites using tooling such as mdBook's `SUMMARY.md` and `book.toml`. [oai_citation:0‡Rust Language](https://rust-lang.github.io/mdBook/guide/creating.html?utm_source=chatgpt.com)

Review Status

Status

Draft

Owner

Lead Design Systems Architect

Next File

`00-document-control.md`

Document Control

Document Metadata

Field	Value
Document ID	MDL-001
Title	Mosaic Design Language Vision
Type	Foundational Specification
Classification	Internal
Status	Draft
Version	0.1
Owner	Lead Design Systems Architect
Repository	`/design/mdl/MDL-001 Vision/`
Parent Standard	DSS-001 Documentation Standard
Target Release	Mosaic v1.0

Purpose

This document establishes the vision of the Mosaic Design Language (MDL).

MDL-001 is the highest-level design specification within the Mosaic documentation hierarchy.

Its purpose is to define **why** Mosaic exists from a design perspective rather than **how** it is implemented.

Every subsequent MDL and MDS specification inherits from this document.

Authority

The authority of this specification extends to:

- User Experience
- Visual Design
- Motion Design
- Information Architecture
- Component Design
- Design Tokens
- Runtime Composition
- Plugin Experience
- Native Client Experience

This document intentionally does **not** prescribe engineering implementation.

Engineering architecture should implement the vision described here, not redefine it.

Design Authority

Changes to MDL-001 require approval from:

Role	Responsibility
Founder	Product vision
Lead Design Systems Architect	Design integrity
Lead Engineer	Technical feasibility

No single role may independently redefine the vision of Mosaic.

Major revisions should be discussed through the Mosaic Design Review process before approval.

Document Lifecycle

```
[mermaid]
flowchart LR

Draft --> Review

Review --> Approved

Approved --> Released

Released --> Superseded
```

Document states are defined as:

Draft

Work in progress.

May change significantly.

Not suitable for implementation.

Review

Submitted for design review.

Only review comments should result in modifications.

Approved

Accepted by the Design Authority.

May be referenced by engineering specifications.

Released

Published as the current authoritative version.

Superseded

Retained for historical reference only.

Must not be used as the basis for future implementation.

Versioning Strategy

MDL specifications follow semantic document versioning.

Version	Meaning
0.x	Draft specification
1.x	Initial approved specification
2.x	Major philosophical revision
x.y.z	Editorial or clarification updates

Minor editorial changes should not alter design intent.

Any modification affecting philosophy, contributor expectations or long-term direction constitutes a major revision.

This approach keeps design intent stable while allowing documentation to evolve through complete revisions rather than fragmented page updates, a practice widely recommended in technical document control. [oai_citation:0‡Hanford Site](https://www.hanford.gov/tocpmm/files.cfm/TFC-ENG-DESIGN-C-25%2C_Technical_Document_Control%2C_Rev__G-19.pdf?utm_source=chatgpt.com)

Revision History

Version	Date	Author	Summary
0.1	July 2026	Lead Design Systems Architect	Initial document created following founder discovery workshops.

Distribution

The latest approved version of this specification shall exist only within the Mosaic repository.

Generated PDFs, printed copies and exported documentation are considered reference copies.

The Markdown source within version control remains the authoritative document.

Dependencies

This specification has no upstream dependencies.

Downstream specifications include:

- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-001 Design Token Architecture
- MDS-002 Material System
- MDS-003 Composition Engine

```
[mermaid]
flowchart TD

MDL001["MDL-001"]

MDL001 --> MDL002
MDL001 --> MDL003
MDL001 --> MDL004
MDL001 --> MDL005

MDL001 --> MDS001
MDL001 --> MDS002
MDL001 --> MDS003
```

Traceability

Every significant design decision introduced by downstream specifications should be traceable back to one or more principles established by MDL-001.

If traceability cannot be demonstrated, either:

1. the proposal belongs outside MDL, or 2. MDL-001 requires formal revision.

This requirement exists to prevent the design language from gradually drifting through isolated implementation decisions.

Review Status

Status

Draft

Outstanding Questions

None.

Next File

`01-background.md`

Background & Problem Statement

Introduction

Every successful design language begins with a clearly understood problem.

Apple's Human Interface Guidelines emerged from the need to make personal computing approachable.

Material Design sought to unify Google's fragmented product ecosystem.

Fluent Design aimed to create consistency across Windows, Office and Xbox.

The Mosaic Design Language exists because **modern entertainment has become fragmented from the user's perspective**.

This document argues that the problem is not the availability of media.

The problem is the experience required to enjoy it.

The State of Entertainment Software

Modern entertainment software has reached an extraordinary level of technical capability.

Users can instantly stream television, films, anime, music and books from almost anywhere.

Yet the overall experience remains fragmented.

A single evening of entertainment might involve:

- continuing a television series
- checking when the next anime episode airs
- reading the manga adaptation
- browsing related artwork
- listening to the soundtrack
- finding the author's next novel
- discussing the series with friends

Although these activities are closely related from the user's perspective, software treats them as unrelated experiences.

The user is repeatedly forced to move between applications, interfaces and interaction models.

Fragmentation Exists at Multiple Layers

Entertainment fragmentation is often described as a problem of services.

Netflix.

Disney+.

Prime Video.

Spotify.

Kindle.

Jellyfin.

Plex.

While this is true, it is only one layer of the problem.

A more fundamental fragmentation exists inside the user's attention.

Every application asks the user to:

- learn new navigation
- understand different terminology
- rebuild spatial memory
- adapt to different interaction models
- mentally switch contexts

The entertainment itself becomes secondary.

The software becomes primary.

Mosaic rejects this relationship.

The Cost of Context Switching

Every interface transition carries cognitive cost.

Even seemingly insignificant actions require users to:

- locate familiar controls
- rebuild orientation
- recall terminology
- understand visual hierarchy
- identify where they were before

Research into media multitasking consistently demonstrates that people switch between digital content extremely frequently, often without consciously recognising how often those interruptions occur. These switches impose measurable cognitive overhead and fragment attention. [oai_citation:0‡PMC](https://pmc.ncbi.nlm.nih.gov/articles/PMC3171998/?utm_source=chatgpt.com)

For Mosaic, every unnecessary context switch is considered design friction.

Reducing friction is therefore a primary design objective rather than an aesthetic preference.

Existing Product Categories

Current entertainment software generally optimises one of four objectives.

Commercial Streaming Platforms

Primary objective:

> *Maximise engagement.*

Characteristics include:

- autoplay
- trending content
- recommendation feeds
- promotion
- retention optimisation

Success is measured by continued consumption.

Media Servers

Primary objective:

> *Organise media collections.*

Characteristics include:

- libraries
- metadata
- transcoding
- administration
- file management

Success is measured by organisation and accessibility.

Collection Managers

Primary objective:

> *Catalogue information.*

These systems excel at recording information but rarely improve the enjoyment of consuming that information.

Companion Applications

Very few products genuinely occupy this space.

A companion application exists to support an activity already chosen by the user.

It assists.

It informs.

It quietly disappears once its work is complete.

Mosaic intentionally positions itself within this category.

The Missing Objective

Existing entertainment software typically optimises one of three outcomes.

- Distribution
- Organisation
- Engagement

Mosaic introduces a fourth.

> ***Immersion***

Immersion becomes the primary optimisation target for every future design decision.

Defining Immersion

Within the Mosaic Design Language, immersion is defined as:

> ***The reduction of mental effort required to remain inside an entertainment experience.***

This definition intentionally extends beyond playback.

Immersion includes:

- watching
- reading
- listening
- discovering
- learning
- continuing
- exploring
- remembering

If software repeatedly demands attention for itself, immersion has been reduced regardless of how visually attractive the interface may be.

Why Mosaic Exists

Mosaic does not exist to become another streaming platform.

It does not exist to become another media server.

It does not exist to become another library manager.

It exists because no current platform is primarily optimised for helping people remain inside the entertainment they have already chosen.

This philosophical distinction influences every subsequent MDL specification.

Design Consequences

Accepting immersion as the primary optimisation target produces several immediate design consequences.

The interface should become quieter as confidence increases.

Artwork should become more expressive.

Navigation should become less prominent.

Movement should explain change rather than attract attention.

Recommendations should deepen the current experience rather than redirect attention elsewhere.

Administration should remain separate from entertainment.

These are not visual preferences.

They are direct consequences of the product vision.

Position Statement

The following statement should guide every future design discussion.

> ***Mosaic is not software for managing media.***
>
> ***Mosaic is software for enjoying media.***

Whenever implementation decisions become unclear, contributors should evaluate whether the proposal strengthens enjoyment or merely strengthens management.

If management becomes more important than enjoyment, the proposal should be reconsidered.

Core Design Challenge

The central challenge addressed by Mosaic can be represented as follows.

```
[mermaid]
flowchart LR

Media["Entertainment"]

Software["Software"]

User["Person"]

Media --> User

Software -.supports.-> Media

Software -.never competes.-> User
```

Software should strengthen the relationship between people and their entertainment.

It should never become the destination itself.

Design Hypothesis

MDL is founded on a single long-term hypothesis.

> *People do not need software that constantly asks for attention.*

They need software that quietly assists until assistance is no longer required.

If this hypothesis proves correct, Mosaic should gradually become less noticeable as it becomes more capable.

That is considered success.

Architectural Implications

The philosophy established within this chapter has direct consequences for future specifications.

Specification	Consequence
MDL-002 Principles	Principles prioritise immersion over engagement.
MDL-003 Mental Model	Users experience a continuous entertainment world rather than isolated applications.
MDL-004 Interaction Model	Interfaces reorganise around changing focus instead of traditional page navigation.
MDL-005 Composition Model	Composition exists to communicate current context while minimising cognitive effort.
MDS Specifications	Visual systems support content rather than competing with it.

Review Status

Status

Draft

Architectural Decisions Introduced

- ADR-004: Mosaic optimises for immersion rather than engagement.
- ADR-005: Context switching is treated as measurable design friction.
- ADR-006: Software exists to support entertainment rather than become the entertainment.

Next File

`02-vision.md`

Vision

Purpose

This chapter defines the long-term vision of Mosaic.

Unlike a mission statement, which explains what an organisation does today, this vision describes the future Mosaic is attempting to create.

It is intentionally aspirational.

It should remain relevant regardless of future implementation details, programming languages, client technologies or architectural changes.

A good product vision acts as a long-lived decision filter rather than a feature list. [oai_citation:0‡IBM](https://www.ibm.com/docs/en/engineering-lifecycle-management-suite/doors-next/beta?topic=requirements-vision-document&utm_source=chatgpt.com)

Vision Statement

> ***Mosaic exists to remove the friction between people and the entertainment they love.***

Entertainment should feel like one continuous experience.

Not a collection of disconnected applications.

Not a collection of disconnected services.

Not a collection of disconnected file formats.

Whether someone is watching a television series, reading a novel, listening to music or exploring an anime franchise, they should remain inside the same entertainment world.

The software should quietly adapt around them.

Long-Term Vision

The long-term ambition of Mosaic is not to become the largest media platform.

Nor is it to become the most feature-rich media server.

The ambition is significantly simpler.

> ***To become the most thoughtful companion for personal entertainment.***

Mosaic should become software that understands what someone is currently enjoying and naturally helps them continue that experience without introducing unnecessary mental effort.

The Future We Are Designing

When Mosaic succeeds:

People stop thinking about applications.

People stop thinking about libraries.

People stop thinking about metadata.

People stop thinking about file formats.

Instead they simply think about:

- what they are enjoying
- what they want to continue
- what they might explore next

The software becomes invisible.

The entertainment becomes memorable.

What Mosaic Is

Mosaic is:

- an entertainment companion
- a contextual experience
- a continuous entertainment world
- a platform that quietly adapts
- software that values immersion over engagement

It exists to strengthen the relationship between people and the media they already love.

What Mosaic Is Not

Mosaic is **not**:

- another streaming service
- another recommendation engine
- another dashboard
- another library manager
- another media server with a different theme

These products optimise different outcomes.

Mosaic intentionally optimises something else.

Immersion.

Core Belief

The vision of Mosaic is founded upon one simple belief.

> ***Personal entertainment should be effortless, allowing anyone to melt into their own private world without friction.***

Everything that follows within the Mosaic Design Language exists to support this belief.

The Companion Philosophy

Mosaic should behave like a knowledgeable companion.

A companion does not dominate a conversation.

A companion does not constantly interrupt.

A companion does not attempt to redirect attention elsewhere.

Instead, a companion quietly supports the experience already taking place.

Examples include:

- reminding someone when the next episode airs
- revealing the novel that inspired a film
- showing the soundtrack currently playing
- helping continue a manga after the anime ends

These interactions deepen the user's existing interest.

They do not attempt to replace it.

Enhancement Over Persuasion

Many modern entertainment platforms optimise for engagement.

They attempt to persuade users to begin something different.

Trending lists.

Featured releases.

Algorithmic promotion.

Infinite feeds.

Mosaic deliberately rejects this model.

Its purpose is not to maximise consumption.

Its purpose is to deepen enjoyment.

The distinction is subtle but fundamental.

Commercial platforms ask:

> *"What should this person watch next?"*

Mosaic asks:

> *"How can we help them enjoy what they've already chosen?"*

The Interface Should Disappear

The greatest compliment Mosaic can receive is not:

> *"That interface is beautiful."*

Instead it is:

> *"I forgot the software was there."*

Once playback begins...

Once reading begins...

Once listening begins...

The interface has completed its primary responsibility.

At that point the entertainment should become the sole focus.

This principle influences every future decision regarding motion, navigation, composition and visual hierarchy.

Relationship to the Roadmap

The long-term technical roadmap describes Mosaic evolving from a compatibility layer into a native entertainment platform with richer metadata, contextual experiences, adaptive composition and first-class support for multiple media domains.

The vision defined by MDL-001 exists independently of those implementation phases.

Whether realised through a Jellyfin compatibility layer, a native GraphQL client or future platform technologies, the objective remains unchanged:

> *Remove friction.*

> *Preserve immersion.*

> *Strengthen the relationship between people and their entertainment.*

Vision Diagram

```
[mermaid]
flowchart TD

Person["Person"]

Entertainment["Entertainment"]

Mosaic["Mosaic Companion"]

Person --> Mosaic

Mosaic --> Entertainment

Entertainment --> Person

style Mosaic fill:#dfe8ff
```

The software exists only to strengthen the connection between the person and their entertainment.

It should never become the destination itself.

Success Criteria

Five years after release, contributors should be able to describe Mosaic without mentioning:

- GraphQL
- Go
- WebAssembly
- HTMX
- Jellyfin
- SQLite
- Components
- Themes

Instead they should describe:

- the experience
- the philosophy
- the feeling

If implementation details become easier to explain than the vision itself, the product has drifted away from its original purpose.

Review Status

Status

Draft

Architectural Decisions Introduced

- ADR-007 — Mosaic is an entertainment companion rather than a media platform.
- ADR-008 — The interface exists to deepen enjoyment, never redirect attention.
- ADR-009 — Success is measured by immersion rather than feature count.

Next File

`03-product-beliefs.md`

Product Beliefs

Purpose

A vision explains **where** Mosaic is going.

Product beliefs explain **how we think**.

These beliefs are intentionally opinionated.

They exist to guide decision making when requirements, user requests or engineering trade-offs become unclear.

Unlike implementation details, product beliefs should remain stable over the lifetime of the platform.

Every future MDL principle, MDS specification and engineering decision should be traceable back to one or more beliefs defined in this chapter. Product visions are most effective when they express enduring beliefs rather than proposed technical solutions. [oai_citation:0‡Hackney Development System](https://playbook.hackney.gov.uk/Product-Playbook/How%20we%20work/product-vision/?utm_source=chatgpt.com)

PB-001

Entertainment Is Personal

Entertainment is one of the most personal activities software can support.

People build emotional connections with:

- films
- television
- anime
- books
- music
- podcasts
- artwork
- characters
- stories

Software should respect those relationships.

It should never attempt to replace them.

Implications

The interface should adapt to the user's interests.

The user should never adapt to the interface.

PB-002

Software Exists To Serve Entertainment

The purpose of Mosaic is not to showcase software.

The purpose of Mosaic is to support entertainment.

Whenever software begins competing for attention with the content itself, the design has failed.

This belief influences:

- visual hierarchy
- motion
- notifications
- recommendations
- overlays
- administration

Design Consequences

Good software becomes quieter over time.

As confidence increases...

The interface should gradually disappear.

PB-003

Context Is More Valuable Than Prediction

Many modern platforms attempt to predict what users may enjoy next.

Prediction is valuable for commercial platforms whose primary objective is engagement.

Mosaic has different objectives.

Mosaic values **context**.

Understanding what someone is currently enjoying produces better experiences than attempting to persuade them towards something different.

Example

Watching:

> *Frieren*

Commercial platform:

> *Trending this week.*

Mosaic:

- Next episode countdown.
- Manga continuation.
- Related soundtrack.
- Cast information.
- Production history.

The user's current interest becomes the centre of the experience.

PB-004

Enhancement Over Persuasion

Mosaic exists to deepen enjoyment.

Not redirect attention.

Recommendations should answer questions like:

- What naturally comes next?
- What belongs here?
- What helps me understand this better?

They should not answer:

> *What keeps the user inside the application longest?*

This distinction separates Mosaic from engagement-driven entertainment platforms.

PB-005

A Companion, Never A Salesperson

The personality of Mosaic is deliberately restrained.

If Mosaic were a person, it would be:

- quiet
- knowledgeable
- friendly
- sophisticated

It would not be:

- demanding
- loud
- promotional
- attention seeking

A good companion speaks when needed.

Then quietly steps aside.

Behaviour

Good companion behaviour:

> *"The next episode releases tomorrow."*

Poor companion behaviour:

> *"Trending now!"*

PB-006

The Interface Should Earn Attention

Every element appearing on screen should justify its existence.

Visual noise creates cognitive noise.

The interface should occupy only the attention necessary to help the user.

When assistance is no longer required, the interface should retreat.

Examples include:

- playback beginning
- reading beginning
- music playback continuing
- long-form viewing

The entertainment becomes primary.

The interface becomes secondary.

PB-007

Respect The User's Current World

People rarely consume media randomly.

They exist inside temporary worlds.

Examples include:

- a particular television series
- an anime season
- an author
- a franchise

- an artist

Mosaic should strengthen the user's current world before encouraging exploration elsewhere.

This creates continuity rather than interruption.

PB-008

Beauty Comes From Restraint

Beautiful interfaces are not created by adding decoration.

They are created by removing unnecessary decisions.

Premium software is:

- calm
- deliberate
- coherent
- predictable

Not because it contains more effects.

Because it contains fewer distractions.

PB-009

Trust Is Earned Through Consistency

People should never wonder:

- why something moved
- why recommendations changed
- why navigation behaves differently
- why information disappeared

Consistency builds trust.

Trust builds confidence.

Confidence allows the interface to disappear.

PB-010

Information Outlives Interfaces

User interfaces evolve.

Technologies evolve.

Frameworks evolve.

The underlying knowledge remains valuable.

Although later MDL specifications formalise this architecture, Mosaic increasingly treats interfaces as temporary expressions of information rather than permanent structures.

This belief encourages separation between:

- knowledge
- relationships
- presentation

allowing future clients and experiences to evolve without redefining the underlying product philosophy.

Summary

The beliefs described within this chapter may be summarised as follows.

```
[mermaid]
mindmap
  root((Mosaic))
    Entertainment is Personal
    Serve Entertainment
    Context over Prediction
    Enhance not Persuade
    Companion
    Earn Attention
    Respect Current World
    Beauty through Restraint
    Trust through Consistency
    Information Outlives Interfaces
```

Together these beliefs establish the philosophical foundation from which every future design principle is derived.

Architectural Decisions

ADR	Decision
ADR-010	Entertainment is considered a personal experience rather than a content catalogue.
ADR-011	Context is preferred over behavioural prediction.
ADR-012	Mosaic behaves as a companion rather than a recommendation engine.
ADR-013	The interface earns attention instead of demanding it.
ADR-014	Long-term architecture should favour information over presentation.

Review Status

Status

Draft

Outstanding Questions

None.

Next File

`04-goals.md`

Goals

Purpose

This chapter defines the long-term goals of the Mosaic Design Language.

Goals describe the outcomes MDL exists to achieve.

They intentionally avoid implementation details.

A goal explains **what success looks like**, not **how success is achieved**. This distinction ensures that future engineering and design teams retain flexibility while remaining aligned with the product vision. [oai_citation:0‡IBM](https://www.ibm.com/docs/en/engineering-lifecycle-management-suite/doors-next/beta?topic=requirements-vision-document&utm_source=chatgpt.com)

Design Goals

Goal 1

Eliminate Entertainment Friction

The primary objective of Mosaic is to reduce the mental effort required to enjoy personal entertainment.

Users should spend less time:

- navigating
- searching
- configuring
- remembering
- organising

and more time:

- watching
- reading
- listening
- discovering
- enjoying

Success is measured by reducing interaction overhead rather than increasing interaction volume.

Goal 2

Preserve Immersion

Every design decision should strengthen immersion.

Immersion is considered broken whenever the interface unnecessarily redirects the user's attention towards itself.

Examples include:

- excessive notifications
- promotional surfaces
- unnecessary configuration
- confusing navigation
- unpredictable movement

Good software should become progressively less noticeable during use.

Goal 3

Create One Entertainment World

Mosaic should feel like one coherent environment regardless of media type.

Television.

Anime.

Books.

Music.

Films.

Future media types.

They should all feel like different parts of the same world rather than different applications sharing the same logo.

Users should never feel they have "changed products."

Only that they have shifted focus.

Goal 4

Build A Trustworthy Companion

Mosaic should become software that users instinctively trust.

Trust is earned through:

- consistency
- predictability
- transparency
- restraint

The interface should never manipulate attention.

It should quietly assist until assistance is no longer required.

Goal 5

Respect Existing Interests

Mosaic should deepen existing interests before introducing new ones.

For example:

Watching an anime should naturally surface:

- upcoming episodes
- manga continuation
- soundtrack
- cast
- related works

rather than immediately attempting to redirect attention towards unrelated content.

The current experience is always considered more valuable than speculative engagement.

Goal 6

Adapt Without Surprising

The interface should continuously adapt to changing context.

However...

Adaptation should never feel unpredictable.

Users should understand:

- why something appeared
- why something disappeared
- why emphasis changed
- why recommendations evolved

Adaptation should strengthen understanding rather than weaken it.

Goal 7

Allow Entertainment To Lead

Media artwork, storytelling and emotional connection should become the primary source of visual identity.

The interface provides:

- structure
- clarity
- consistency

The entertainment provides:

- emotion
- atmosphere
- identity

Mosaic should never compete with the media it presents.

Goal 8

Build For Evolution

The vision of Mosaic should remain stable even as technology changes.

Future changes may include:

- new rendering engines
- new client platforms
- new interaction models
- new plugin systems
- new AI capabilities

These should evolve beneath the design language rather than forcing the design language to change.

MDL exists specifically to provide this long-term stability.

Product Goals

The following goals describe what the completed Mosaic experience should achieve for users.

Goal	Outcome
Reduce friction	Users spend less time managing software.
Preserve immersion	Users remain focused on entertainment.
Increase continuity	Media formats feel connected.
Strengthen trust	Behaviour becomes predictable.
Encourage exploration	Users naturally discover deeper relationships within their existing interests.

Engineering Goals

Although MDL is not an engineering specification, it establishes expectations that influence architecture.

Future systems should enable:

- adaptive composition
- context-aware presentation
- runtime atmosphere

- extensibility
- accessibility
- device independence

Engineering solutions should support these goals without redefining them.

Design Success Metrics

The success of MDL should not be measured through traditional UI metrics alone.

Instead, contributors should ask questions such as:

- Does the interface require fewer conscious decisions?
- Does the user remain immersed longer?
- Does navigation become easier over time?
- Does the software quietly fade into the background?
- Does the current context feel respected?

Positive answers indicate progress towards the design goals.

Goal Relationships

```
[mermaid]
flowchart TD

Vision["Vision"]

Vision --> Friction["Reduce Friction"]

Vision --> Immersion["Preserve Immersion"]

Vision --> Trust["Build Trust"]

Vision --> Context["Respect Context"]

Vision --> World["One Entertainment World"]

Friction --> Companion["Trusted Companion"]

Immersion --> Companion

Trust --> Companion

Context --> Companion

World --> Companion
```

All design goals ultimately contribute towards a single outcome.

A trusted entertainment companion.

Design Trade-offs

When goals conflict, they should be prioritised in the following order.

1. Preserve immersion. 2. Reduce friction. 3. Respect current context. 4. Maintain consistency. 5. Introduce new capabilities.

This ordering intentionally favours experience quality over feature quantity.

Review Status

Status

Draft

Architectural Decisions Introduced

- ADR-015 — Reducing friction is the primary measure of design success.
- ADR-016 — Entertainment always has priority over interface expression.
- ADR-017 — New capabilities must strengthen, not weaken, immersion.

Next File

`05-non-goals.md`

Non-Goals

Purpose

A strong vision is defined as much by what it refuses to become as by what it aspires to achieve.

Non-goals exist to protect the long-term identity of Mosaic.

Every successful platform eventually encounters feature requests, architectural proposals and community contributions that appear individually reasonable but collectively move the product away from its original purpose.

This chapter establishes the boundaries of the Mosaic Design Language.

These are deliberate decisions.

They are not limitations.

Research and industry guidance consistently recommend documenting explicit non-goals because they reduce scope creep and provide a shared framework for rejecting ideas that fall outside the product vision. [oai_citation:0‡Courses at Washington](https://courses.cs.washington.edu/courses/cse403/24au/lectures/04-requirements.pdf?utm_source=chatgpt.com)

NG-001

Mosaic Is Not A Streaming Service

Mosaic does not exist to compete with commercial streaming platforms.

It is not responsible for:

- licensing content
- promoting content
- producing original content
- maximising viewing hours
- advertising releases

The software exists to help people enjoy **their** entertainment.

Not ours.

NG-002

Mosaic Does Not Optimise Engagement

Many entertainment platforms optimise:

- daily active users
- watch time

- session length
- recommendation click-through
- autoplay completion

These are valid commercial objectives.

They are not Mosaic objectives.

Mosaic optimises:

- immersion
- continuity
- comfort
- understanding
- trust

These metrics may occasionally conflict.

When they do, immersion takes priority.

NG-003

Mosaic Does Not Tell People What They Should Enjoy

The interface should never attempt to replace the user's interests with its own.

Examples of behaviour Mosaic intentionally avoids:

- "Trending Now"
- "Because everyone else watched..."
- "Editor's Picks"
- "Popular This Week"

These may exist as optional extensions or community plugins.

They are not part of the core Mosaic experience.

The core platform always begins with the user's current context.

NG-004

Mosaic Is Not A Dashboard

Dashboards optimise visibility.

Mosaic optimises understanding.

A dashboard attempts to display everything simultaneously.

Mosaic deliberately hides complexity until it becomes useful.

The interface should reveal information progressively as context changes.

Users should never feel overwhelmed simply because more information exists.

NG-005

Mosaic Does Not Reward Complexity

Additional features do not automatically improve the experience.

A proposal that adds capability while increasing friction should be viewed critically.

The preferred solution is often:

- fewer controls
- fewer screens
- fewer decisions
- fewer interruptions

Design maturity is measured by thoughtful restraint rather than feature quantity.

NG-006

Mosaic Is Not Built Around Pages

Traditional applications organise experiences around navigation.

```
Home
↓
Library
↓
Series
↓
Season
↓
Episode
```

Mosaic intentionally moves away from this model.

Future MDL specifications define the interface as a continuously evolving composition rather than a sequence of pages.

This distinction influences every later specification but is introduced here to establish intent.

NG-007

Mosaic Does Not Expose Internal Architecture

Users should never need to understand:

- GraphQL
- Jellyfin compatibility
- plugin boundaries
- storage engines
- metadata providers
- caching strategies

Implementation details belong to engineering.

The user experience should remain independent from technical architecture.

NG-008

Mosaic Does Not Allow Visual Competition

The interface should never compete with entertainment artwork.

Artwork communicates:

- emotion
- identity
- atmosphere

The interface communicates:

- structure
- hierarchy
- continuity

If interface chrome becomes more visually interesting than the content itself, the design has failed.

NG-009

Mosaic Is Not A Recommendation Engine

Recommendations are a consequence of context.

They are not the purpose of the platform.

Mosaic should avoid behaviour that attempts to maximise engagement through algorithmic persuasion.

Instead, recommendations should answer questions such as:

- What naturally comes next?
- What belongs with this?
- What completes this experience?

NG-010

Mosaic Does Not Sacrifice Clarity For Novelty

Innovation is encouraged.

Novelty is not.

Every interaction must remain immediately understandable.

If a new interaction requires explanation before it becomes usable, it should be reconsidered.

Originality is valuable only when it improves understanding.

Trade-off Philosophy

Whenever uncertainty exists, contributors should ask:

> ***Does this make Mosaic more like a companion... or more like a platform?***

If the proposal pushes Mosaic towards becoming another platform competing for attention, it should be reconsidered.

If the proposal strengthens Mosaic as a trusted companion, it is likely aligned with the vision.

Examples

Good

A user finishes an anime episode.

Mosaic presents:

- the next episode release date
- manga continuation
- soundtrack
- production notes

The current experience is deepened.

Poor

A user finishes an anime episode.

Mosaic replaces the screen with:

- trending releases
- unrelated recommendations
- promoted content
- autoplay trailers

The user's attention has been redirected away from their original interest.

Relationship To Future Specifications

These non-goals establish constraints for every downstream MDL and MDS document.

Future specifications should not introduce behaviour that contradicts these boundaries without first proposing a formal amendment to MDL-001.

```
[mermaid]
flowchart TD

Vision --> Goals

Goals --> NonGoals

NonGoals --> Principles

Principles --> DesignSystem

DesignSystem --> Implementation
```

Non-goals are not restrictions on creativity.

They are protections for product identity.

Architectural Decisions

ADR	Decision
ADR-018	Engagement optimisation is explicitly outside the scope of Mosaic.
ADR-019	The interface must never become more important than the entertainment.
ADR-020	Internal architecture must remain invisible to users.
ADR-021	Product identity is protected through explicit non-goals.

Review Status

Status

Draft

Outstanding Questions

None.

Next File

```
06-design-philosophy.md
```

Design Philosophy

Purpose

This chapter describes the philosophy that underpins every design decision within Mosaic.

Unlike principles, which help contributors choose between competing solutions, philosophy explains the worldview from which those principles emerge.

It answers one question:

> ***How should Mosaic think?***

Every future specification should be able to trace its reasoning back to the philosophy established here.

Design Philosophy Statement

> ***Mosaic is a trusted entertainment companion that quietly adapts around people, allowing them to disappear into the worlds they love.***

This statement represents the essence of the Mosaic Design Language.

Everything else is an implementation.

Philosophy Before Interface

Most software begins by designing screens.

Mosaic begins by designing relationships.

Relationships between:

- people
- entertainment
- information
- context
- understanding

The interface exists only to communicate those relationships.

The interface itself is never the product.

People Before Technology

Technology changes continuously.

Programming languages evolve.

Rendering engines evolve.

Frameworks evolve.

Interaction devices evolve.

Human behaviour evolves much more slowly.

MDL therefore places people before technology.

The question is never:

> *"What can this technology do?"*

The question is:

> *"What experience are we trying to create?"*

Engineering decisions should support experience.

Experience should never be dictated by engineering.

Entertainment Before Software

People do not open Mosaic because they enjoy software.

They open Mosaic because they enjoy entertainment.

This distinction sounds obvious.

It is also surprisingly easy to forget.

Whenever software becomes more memorable than the entertainment it presents, the design has failed.

The interface should gradually disappear as confidence increases.

Entertainment should become increasingly visible.

Context Before Prediction

Many digital products attempt to understand users through prediction.

Prediction attempts to answer:

> *What might this person want next?*

Mosaic begins somewhere simpler.

Context asks:

> *What is this person enjoying right now?*

Current context is considered more valuable than speculative future behaviour.

This philosophy influences:

- recommendations
- navigation
- composition
- notifications

- plugins

Understanding should always precede prediction.

Enhancement Before Persuasion

Commercial platforms often optimise attention.

Mosaic optimises enjoyment.

This creates a fundamentally different philosophy.

Instead of asking:

> *How do we keep someone inside the application?*

Mosaic asks:

> *How do we improve the experience they have already chosen?*

The distinction may appear subtle.

It fundamentally changes product behaviour.

Examples include:

Watching a television series.

Commercial response:

- trending releases
- featured content
- promoted originals

Mosaic response:

- next episode
- soundtrack
- source novel
- cast
- production history

One redirects.

The other deepens.

Calm Before Excitement

Excitement is valuable.

Constant excitement is exhausting.

Mosaic should feel calm.

Not because calm is visually minimalist.

Because calm respects attention.

The interface should avoid:

- unnecessary urgency
- excessive visual noise
- competing animations
- promotional hierarchy

The software should communicate confidence through restraint.

Good design principles are intended to guide trade-offs rather than act as slogans. They provide teams with a consistent basis for choosing one solution over another. [oai_citation:0‡Design Principles](https://principles.design/about/?utm_source=chatgpt.com)

Consistency Before Cleverness

Novel interactions are encouraged.

Confusing interactions are not.

Originality should emerge from solving problems differently.

Not from inventing unfamiliar behaviours.

Whenever contributors propose something innovative they should ask:

> *Does this improve understanding?*

If the answer is no, novelty alone is insufficient.

Systems Before Features

Features solve individual problems.

Systems solve categories of problems.

Mosaic intentionally invests in systems.

Examples include:

- adaptive composition
- runtime atmosphere
- information-driven presentation
- contextual navigation

These systems allow future features to emerge naturally instead of being individually designed.

A feature should strengthen an existing system before introducing a new one.

Evolution Before Reinvention

The Mosaic Design Language is expected to evolve.

It is not expected to restart.

Future revisions should build upon existing philosophy rather than replacing it.

Large philosophical changes require evidence that the existing philosophy no longer serves users.

This creates continuity across decades rather than product cycles.

The Companion Model

The personality of Mosaic may be summarised through one metaphor.

Mosaic is the knowledgeable friend sitting beside you on the sofa.

That friend:

- knows what you're watching
- remembers where you stopped
- knows when the next episode releases
- remembers which author wrote the novel
- knows which soundtrack is currently playing

They quietly mention useful information.

Then they stop talking.

They do not attempt to dominate the evening.

This metaphor should guide every future design discussion.

Whenever uncertainty exists contributors should ask:

> ***Would a trusted companion behave like this?***

If the answer is no, the proposal should be reconsidered.

Design Philosophy Model

```
[mermaid]
flowchart TD

People --> Entertainment

Entertainment --> Context

Context --> Understanding

Understanding --> Companion

Companion --> Experience

Experience --> Immersion
```

Every layer exists to strengthen the relationship between people and entertainment.

The interface is simply the mechanism through which that relationship is expressed.

Philosophy Summary

The Mosaic Design Language may be summarised through six philosophical statements.

1. People before technology. 2. Entertainment before software. 3. Context before prediction. 4. Enhancement before persuasion. 5. Calm before excitement. 6. Systems before features.

Every downstream MDL and MDS specification should reinforce these ideas.

Architectural Decisions

ADR	Decision
ADR-022	Experience always takes precedence over implementation technology.
ADR-023	Context is considered a more reliable design foundation than behavioural prediction.
ADR-024	Mosaic behaves as a companion rather than an engagement platform.
ADR-025	New systems are preferred over isolated feature implementations.

Review Status

Status

Draft

Outstanding Questions

None.

Next File

`07-governance.md`

Governance

Purpose

A design language only remains valuable if it is actively governed.

Without governance, design systems inevitably drift as individual features optimise for local requirements instead of the long-term product vision.

The purpose of governance is **not** to slow development.

Its purpose is to preserve coherence while allowing the product to evolve.

Philosophy

Governance exists to protect the user experience.

It does **not** exist to protect individual design decisions.

Every contributor should feel empowered to challenge existing patterns when evidence suggests a better solution exists.

However, changes should always strengthen the Mosaic Design Language rather than fragment it.

The role of governance is therefore:

- preserve consistency
- encourage evolution
- document reasoning
- avoid accidental drift

Governance Principles

The governance model for MDL is built upon five principles.

1. Philosophy Before Implementation

Implementation details are expected to evolve.

Philosophy should remain comparatively stable.

Whenever implementation and philosophy conflict, implementation should change first.

2. Decisions Must Be Traceable

Every significant design decision should be traceable to:

- Vision
- Principles

- Mental Model
- Interaction Model
- Composition Model

No significant design decision should exist without rationale.

Architectural Decision Records (ADRs) are widely recommended as a lightweight mechanism for preserving the context and consequences of important design decisions over time. [oai_citation:0‡Google Cloud Documentation](https://docs.cloud.google.com/architecture/architecture-decision-records?hl=en&utm_source=chatgpt.com)

3. Evolution Over Reinvention

MDL is expected to evolve.

It is not expected to restart.

Future contributors should improve existing systems before inventing new ones.

Large philosophical changes require:

- evidence
- rationale
- documented consequences
- formal review

4. Repository Is The Source Of Truth

The Markdown documents stored within the Mosaic repository are the authoritative version of MDL.

Generated documentation is considered a publication artefact.

Examples include:

- PDF
- HTML
- Documentation website
- Printed copies

These should never be edited directly.

5. Every Decision Leaves A Trail

Whenever MDL changes:

The reason should be documented.

Future contributors should never need to guess:

- why a decision exists
- why alternatives were rejected

- why behaviour changed

Historical context is considered part of the design language.

Governance Model

```
[mermaid]
flowchart TD

Proposal["Proposal"]

Proposal --> Discussion

Discussion --> ADR

ADR --> Review

Review --> Approval

Approval --> Specification

Specification --> Implementation

Implementation --> Validation
```

No design decision should move directly from idea to implementation.

Design Authority

The following roles participate in MDL governance.

Role	Responsibility
Founder	Product vision
Lead Design Systems Architect	Design integrity
Lead Engineer	Technical feasibility
Contributors	Proposals and implementation
Community	Feedback and validation

Responsibility is intentionally distributed.

No single individual owns the design language indefinitely.

Decision Types

Not every design decision requires the same level of review.

Editorial

Examples:

- spelling
- wording

- clarification

Review:

Repository pull request.

Design

Examples:

- interaction changes
- component behaviour
- terminology

Review:

Design review required.

Philosophical

Examples:

- changing product beliefs
- changing principles
- redefining immersion
- altering companion philosophy

Review:

Founder approval required.

ADR required.

Version increment required.

Design Review Process

Every significant proposal should answer the following questions.

Vision

Does this strengthen the long-term vision?

Philosophy

Does this align with the companion philosophy?

Principles

Which MDL principles does this reinforce?

Which principles become weaker?

User Experience

Does this reduce friction?

Does this preserve immersion?

Does this respect the user's current context?

Future Evolution

Does this make future design simpler...

...or more complicated?

Design Review Lifecycle

```
[mermaid]
stateDiagram-v2

[*] --> Proposed

Proposed --> UnderReview

UnderReview --> Accepted

UnderReview --> RevisionRequired

RevisionRequired --> UnderReview

Accepted --> Implemented

Implemented --> Validated

Validated --> Living
```

Design Debt

Technical debt accumulates within software.

Design debt accumulates within experiences.

Examples include:

- duplicated interaction patterns
- inconsistent terminology
- competing navigation models
- unnecessary visual complexity
- exceptions without rationale

Design debt should be treated as seriously as technical debt.

Future specifications may introduce dedicated Design Debt Reviews.

Specification Ownership

Each MDL specification has a designated owner.

Ownership exists to maintain quality.

Ownership does **not** imply exclusive authorship.

Contributors are encouraged to propose improvements through the normal review process.

Success Criteria

Governance succeeds when:

- contributors make similar decisions independently
- specifications rarely contradict one another
- new features feel inevitable rather than bolted on
- historical decisions remain understandable years later
- the product evolves without losing its identity

Architectural Decisions

ADR	Decision
ADR-026	Markdown specifications stored in version control are the authoritative source of MDL.
ADR-027	Significant philosophical changes require formal review and ADRs.
ADR-028	Governance exists to preserve product identity rather than individual implementation details.
ADR-029	Design debt should be tracked and reviewed alongside technical debt.

Review Status

Status

Draft

Outstanding Questions

None.

Next File

`08-adrs.md`

Architectural Decision Records

Purpose

Architectural Decision Records (ADRs) preserve the reasoning behind significant design decisions.

MDL is intended to guide Mosaic for many years.

Without recorded rationale, future contributors are forced to rediscover the reasoning behind decisions or, worse, unknowingly reverse them.

Every design decision recorded here should answer four questions:

1. What problem were we solving? 2. What decision was made? 3. Why was this decision chosen? 4. What are the long-term consequences?

This approach aligns with widely adopted ADR practices, where each decision captures its context, decision and consequences to preserve architectural knowledge over time. [oai_citation:0‡AWS Documentation](https://docs.aws.amazon.com/prescriptive-guidance/latest/architectural-decision-records/adr-process.html?utm_source=chatgpt.com)

ADR Format

Every future MDL and MDS ADR should follow the same structure.

```
[text]
ADR-XXX

Status

Context

Decision

Consequences

Alternatives Considered

Related Specifications
```

One ADR should describe one significant decision.

Multiple unrelated decisions should never be combined into a single record.

ADR-001

Title

Treat Mosaic as an Entertainment Companion rather than a Media Server.

Status

Accepted

Context

Traditional media servers optimise organisation.

Commercial streaming services optimise engagement.

Neither philosophy reflects the long-term vision established during founder discovery.

Decision

Mosaic is defined as an entertainment companion.

The interface exists to support a person's current entertainment experience rather than encourage unrelated consumption.

Consequences

Future specifications should favour:

- assistance
- continuity
- immersion

over:

- promotion
- engagement
- feature visibility

Alternatives Considered

- Media Server
- Streaming Platform
- Content Dashboard

Rejected because each encourages software to become the centre of attention.

Related Specifications

- MDL-001 Vision
- MDL-002 Principles

ADR-002

Title

Optimise for Immersion rather than Engagement.

Status

Accepted

Context

Most entertainment software measures success using engagement metrics.

These objectives conflict with the desired Mosaic experience.

Decision

Immersion becomes the primary optimisation target.

Consequences

Future features should be evaluated according to whether they reduce friction and preserve attention.

Feature popularity alone is insufficient justification.

Alternatives Considered

- Session length
- Watch time
- Click-through rate

Rejected because they optimise commercial outcomes rather than user experience.

ADR-003

Title

Context Is More Valuable Than Prediction.

Status

Accepted

Context

Recommendation engines generally attempt to predict future behaviour.

Founder workshops consistently identified current context as significantly more valuable.

Decision

Mosaic should first understand what the user is currently doing.

Only then should it offer additional information.

Consequences

Examples include:

Watching anime:

- next episode
- manga continuation
- soundtrack

instead of:

- unrelated trending content

Related Specifications

MDL-003 Mental Model

MDL-004 Interaction Model

ADR-004

Title

The Interface Should Gradually Disappear.

Status

Accepted

Context

Users begin Mosaic because they wish to enjoy entertainment.

Not software.

Decision

The interface should occupy attention only while it remains useful.

Once playback, reading or listening begins, interface chrome should progressively reduce its visual prominence.

Consequences

Future MDS specifications should favour:

- restrained motion
- restrained overlays
- contextual controls
- artwork-first presentation

ADR-005

Title

Artwork Provides Emotion.

Status

Accepted

Context

Media already possesses a rich emotional identity through artwork.

Attempting to compete with that identity weakens both the content and the interface.

Decision

Artwork becomes the primary source of emotional expression.

The interface provides:

- hierarchy
- clarity
- consistency

rather than emotional emphasis.

Consequences

Future material and colour systems derive atmosphere from artwork while preserving a consistent Mosaic brand identity.

ADR-006

Title

The User's Current World Is The Primary Design Unit.

Status

Accepted

Context

Traditional applications organise themselves around pages.

Founder discovery established that people experience entertainment as connected worlds rather than isolated screens.

Decision

Future specifications should organise experiences around the user's current world.

Navigation exists only to support changing focus.

Consequences

This ADR directly motivates:

- Composition Model
- Interaction Model
- Runtime Composition Engine

ADR-007

Title

Build Systems Before Features.

Status

Accepted

Context

Feature-driven products accumulate inconsistency over time.

Systems allow future capabilities to emerge naturally.

Decision

Where possible, Mosaic should invest in reusable systems rather than isolated features.

Examples include:

- adaptive composition
- runtime atmosphere
- information-driven presentation

Consequences

Engineering effort may initially increase.

Long-term complexity decreases.

ADR Relationships

```
[mermaid]
flowchart TD

ADR001["Companion"]

ADR002["Immersion"]

ADR003["Context"]

ADR004["Disappear"]

ADR005["Artwork"]

ADR006["World"]

ADR007["Systems"]

ADR001 --> ADR002

ADR002 --> ADR003

ADR003 --> ADR006

ADR005 --> ADR004

ADR006 --> ADR007
```

Architectural decisions are intentionally connected.

No ADR should exist in isolation.

Future ADRs

This document intentionally records only the foundational decisions required to establish the Mosaic Design Language.

Future MDL and MDS specifications are expected to introduce additional ADRs covering:

- terminology
- composition
- motion
- materials
- token architecture
- runtime systems
- extension model
- accessibility
- information architecture

Each ADR should remain focused on a single architecturally significant decision.

ADR Governance

An ADR may be:

- Proposed
- Accepted
- Superseded
- Deprecated

Superseded ADRs should never be deleted.

Instead they should reference the ADR that replaces them.

Maintaining this decision history helps future contributors understand how and why the design language evolved, a common recommendation in established ADR practices. [oai_citation:1‡github.com](https://github.com/architecture-decision-record/architecture-decision-record?utm_source=chatgpt.com)

Review Status

Status

Draft

Outstanding Questions

None.

Next File

`09-contributor-guidance.md`

Contributor Guidance

Purpose

The Mosaic Design Language is intended to outlive individual contributors.

This chapter exists to ensure that every designer, engineer and community contributor approaches the product using the same mental model.

It is not a coding standard.

It is not a visual style guide.

It is guidance for making decisions.

Good design system documentation explains not only *what* exists, but *why*, *when* and *how* contributors should use it, reducing inconsistency as teams grow. [oai_citation:0‡Magic Patterns](https://www.magicpatterns.com/blog/design-system-documentation?utm_source=chatgpt.com)

The Responsibility Of Every Contributor

Every contribution changes the experience of using Mosaic.

Whether writing backend services, designing interactions or implementing components, contributors are responsible for protecting the philosophy established by MDL.

Every contribution should leave the product:

- simpler
- calmer
- easier to understand
- more immersive
- more coherent

No contribution should exist purely because it is technically possible.

The Contributor Mindset

Before beginning implementation, contributors should understand one fundamental truth.

> ***You are not building screens.***

You are building an entertainment companion.

Everything else follows from that statement.

Five Questions

Before opening a Pull Request, every contributor should answer the following questions.

1.

Does this reduce friction?

If the proposal introduces additional steps, navigation or mental effort, reconsider the design.

2.

Does this preserve immersion?

Will someone remain focused on their entertainment...

...or will they become focused on the software?

3.

Does this respect the user's current context?

Does the proposal strengthen what the user is already doing...

...or attempt to redirect them somewhere else?

4.

Does this strengthen existing systems?

Can this capability be expressed using:

- existing composition rules
- existing interaction patterns
- existing terminology
- existing materials

or is it introducing unnecessary exceptions?

5.

Would a trusted companion behave this way?

This question should become instinctive.

If the answer is:

"No."

The proposal almost certainly requires redesign.

Contribution Hierarchy

When making decisions, contributors should follow this hierarchy.

```
[mermaid]
flowchart TD
```

```
Vision

Vision --> Philosophy

Philosophy --> Principles

Principles --> MentalModel

MentalModel --> Interaction

Interaction --> Composition

Composition --> DesignSystem

DesignSystem --> Implementation
```

Contributors should avoid solving disagreements by introducing implementation-specific arguments before checking the higher levels of the hierarchy.

What Good Contributions Look Like

Good contributions typically:

- simplify existing interactions
- strengthen consistency
- reduce cognitive effort
- remove unnecessary options
- improve accessibility
- reuse existing systems
- make future work easier

Good contributions often remove more than they add.

What Poor Contributions Look Like

Poor contributions often:

- introduce competing interaction models
- duplicate existing capabilities
- expose implementation details
- prioritise novelty over clarity
- increase visual noise
- increase configuration complexity
- solve one feature while weakening the overall experience

These proposals should be challenged regardless of implementation quality.

Designing New Features

When proposing a new feature, contributors should avoid asking:

> *"Where should this screen go?"*

Instead ask:

> *"What problem is the user experiencing?"*

Then ask:

> *"Can the existing Mosaic systems solve this naturally?"*

New systems should be introduced only when existing systems genuinely cannot support the required experience.

Extending Rather Than Replacing

Every new contribution should attempt to strengthen an existing system before creating another.

Examples include:

Prefer:

- extending composition rules

instead of:

- introducing a new layout model

Prefer:

- extending tile behaviour

instead of:

- inventing a new visual primitive

Prefer:

- extending interaction patterns

instead of:

- introducing isolated behaviours

This philosophy reduces long-term design fragmentation.

Language Matters

Contributors should use MDL terminology consistently.

For example:

Avoid	Prefer
Screen	Composition
Dashboard	World / Composition

Avoid	Prefer
Widget	Tile (until superseded by future Information Model)
Recommendation Engine	Companion
Navigation	Focus Change

Consistent language produces consistent thinking.

Future specifications define this vocabulary formally.

Design Reviews

Every significant proposal should include answers to the following questions.

- Which MDL principle does this reinforce?
- Which user problem does this solve?
- Does it introduce additional friction?
- Does it reduce immersion?
- Can the same outcome be achieved using an existing system?
- Does this strengthen Mosaic as a companion?

Review comments should reference MDL sections rather than personal preference whenever possible.

Community Contributions

Community contributions are encouraged.

However, acceptance is determined by alignment with MDL rather than popularity.

A proposal should demonstrate that it:

- serves a genuine user need
- aligns with established philosophy
- reuses existing systems where possible
- does not duplicate existing capabilities

These contribution criteria mirror the governance models used by mature design systems, where proposals must demonstrate usefulness, consistency and broad applicability before becoming part of the core system. [oai_citation:1‡GOV.UK Design System](https://design-system.service.gov.uk/community/contribution-criteria/?utm_source=chatgpt.com)

A Simple Rule

If contributors remember only one sentence from this chapter, it should be this:

> ***Build experiences people forget they are using.***

People should remember:

- the story
- the music
- the characters
- the artwork

They should rarely remember the interface.

That is considered success.

Architectural Decisions

ADR	Decision
ADR-030	Contributors should solve user problems before proposing interface changes.
ADR-031	Existing systems should be strengthened before new systems are introduced.
ADR-032	Design discussions should reference MDL rather than personal preference.
ADR-033	Community contributions are evaluated by alignment with MDL, not popularity.

Review Status

Status

Draft

Outstanding Questions

The formal terminology table will be expanded by **MDL-005 Vocabulary**.

Next File

`10-design-review-checklist.md`

Design Review Checklist

Purpose

The purpose of this checklist is to provide a consistent framework for reviewing design proposals against the Mosaic Design Language.

The checklist exists to remove subjectivity from design discussions.

Rather than asking:

> *"Do we like this?"*

Reviewers should ask:

> *"Does this align with MDL?"*

Every significant feature, interaction, component, motion system or architectural proposal should be reviewed against this checklist before implementation.

Structured design review checklists are widely used because they improve consistency, reduce omissions and encourage objective discussions over subjective preference. [oai_citation:0‡Smartsheet](https://www.smartsheet.com/content/design-review-checklist-templates?utm_source=chatgpt.com)

Review Philosophy

Design reviews exist to answer one question.

> ***Does this proposal make Mosaic feel more like Mosaic?***

Reviews are not intended to:

- approve personal preferences
- optimise visual novelty
- reward technical complexity

Instead they protect the long-term integrity of the design language.

Review Levels

Every proposal should be classified before review.

Level	Examples	Review Required
Editorial	Documentation, wording	Maintainer
Minor Design	Small UI improvement	Design Review
Major Design	New interaction or component	Design Review + Founder
Philosophical	Changes to MDL	Design Authority

The higher the impact on the product identity, the higher the level of review required.

Section A

Vision

A1

Does the proposal strengthen the vision established in MDL-001?

- [ ] Yes
- [ ] No

A2

Does it reduce friction?

- [ ] Yes
- [ ] No

A3

Does it preserve immersion?

- [ ] Yes
- [ ] No

A4

Does it strengthen Mosaic as an entertainment companion?

- [ ] Yes
- [ ] No

Section B

Product Beliefs

B1

Does the proposal respect the user's current context?

- [ ] Yes
- [ ] No

B2

Does it deepen the current experience rather than redirect attention?

- [ ] Yes

- [ ] No

B3

Would a knowledgeable companion behave this way?

- [ ] Yes
- [ ] No

B4

Does the proposal earn the user's attention rather than demand it?

- [ ] Yes
- [ ] No

Section C

User Experience

C1

Can the user understand what changed?

- [ ] Yes
- [ ] No

C2

Is the interface calmer after this change?

- [ ] Yes
- [ ] No

C3

Has unnecessary cognitive effort been removed?

- [ ] Yes
- [ ] No

C4

Is artwork still the primary emotional focus?

- [ ] Yes
- [ ] No

Section D

Systems

D1

Does the proposal reuse an existing MDL or MDS system?

- [ ] Yes
- [ ] No

D2

If a new system is introduced, has it been justified?

- [ ] Yes
- [ ] No

D3

Can this capability be expressed through existing composition rules?

- [ ] Yes
- [ ] No

D4

Does this reduce long-term design complexity?

- [ ] Yes
- [ ] No

Section E

Engineering

E1

Does the proposal expose implementation details to users?

- [ ] Yes
- [ ] No

If **Yes**, redesign.

E2

Does the proposal remain platform independent?

- [ ] Yes

- [ ] No

E3

Will this behave consistently across supported devices?

- [ ] Yes
- [ ] No

E4

Can future extensions participate naturally?

- [ ] Yes
- [ ] No

Section F

Accessibility

F1

Does this improve accessibility?

- [ ] Yes
- [ ] No

F2

Does movement remain understandable with reduced motion enabled?

- [ ] Yes
- [ ] No

F3

Can this interaction be completed without relying on animation?

- [ ] Yes
- [ ] No

F4

Does this preserve readability and visual hierarchy?

- [ ] Yes
- [ ] No

Automatic Rejection Criteria

A proposal should normally be rejected if it:

- introduces unnecessary friction
- competes with entertainment for attention
- exposes implementation details
- duplicates an existing system
- weakens established MDL principles
- requires explanation before becoming understandable
- prioritises novelty over clarity

These criteria intentionally bias Mosaic towards restraint rather than feature accumulation.

Review Outcome

Every review should conclude with one of the following outcomes.

Outcome	Meaning
Accepted	Ready for implementation
Accepted with Revisions	Minor changes required
Deferred	Requires additional investigation
Rejected	Conflicts with MDL
Superseded	Proposal replaced by another approach

Rejected proposals should always reference the relevant MDL section or ADR rather than relying on subjective reasoning.

Review Record

Every completed review should record:

```
[text]
Specification:

Reviewer(s):

Date:

Decision:

Relevant ADRs:

Relevant MDL Sections:

Follow-up Actions:

Review Notes:
```

This creates a permanent audit trail explaining why significant design decisions were accepted or rejected.

One Final Question

Before approving any proposal, every reviewer should ask:

> ***If this became the standard across every part of Mosaic, would the product become stronger or weaker?***

If the answer is uncertain, the proposal should remain in review.

Architectural Decisions

ADR	Decision
ADR-034	Design reviews evaluate alignment with MDL rather than personal preference.
ADR-035	Every significant proposal must leave a documented review trail.
ADR-036	Existing systems are preferred over introducing isolated exceptions.

Review Status

Status

Draft

Outstanding Questions

None.

Next File

`11-future-considerations.md`

Future Considerations

Purpose

MDL-001 intentionally defines a vision rather than a complete solution.

As Mosaic evolves, new technologies, interaction models and user expectations will emerge.

This chapter documents areas that are expected to evolve while preserving the philosophy established by MDL-001.

Future considerations are **not commitments**.

They are strategic directions that should be explored when they strengthen the core vision.

A mature design language should provide long-term direction while remaining flexible enough to accommodate future implementation and organisational changes. [oai_citation:0‡U.S. Web Design System (USWDS)](https://designsystem.digital.gov/next/looking-ahead/vision/?utm_source=chatgpt.com)

Guiding Principle

The philosophy of Mosaic should remain significantly more stable than its implementation.

Over the lifetime of the project we expect:

- programming languages to change
- frameworks to change
- rendering engines to change
- devices to change
- interaction models to change

The experience people have when using Mosaic should remain recognisably Mosaic.

Future Direction 01

Information-Driven Experiences

Current software generally renders interfaces first.

Information is inserted afterwards.

Long-term, Mosaic should move towards the opposite model.

```
Knowledge

↓

Relationships

↓

Composition
```

↓

```
Presentation
```

Rather than asking:

> *Which widget should we display?*

Future systems should ask:

> *What information best helps the user right now?*

This distinction may ultimately influence:

- GraphQL
- extension APIs
- runtime composition
- adaptive interfaces

This concept is intentionally deferred to future MDL and MDS specifications.

Future Direction 02

Adaptive Composition

Traditional applications navigate between pages.

Mosaic should continue exploring adaptive composition.

Future research areas include:

- context-aware layouts
- relationship-driven composition
- adaptive emphasis
- progressive disclosure
- composition solving

The objective is not novelty.

The objective is reducing cognitive effort.

Future Direction 03

Rich Relationships

Entertainment consists of relationships rather than isolated media.

Examples include:

- books adapted into films
- anime adapted from manga

- actors appearing across franchises
- composers contributing to multiple works
- directors influencing visual style

Future Mosaic experiences should increasingly surface these relationships naturally.

Users should discover more about what they already enjoy.

Not simply more content.

Future Direction 04

Atmosphere

Artwork already provides emotional context.

Future material systems should continue exploring:

- contextual atmosphere
- adaptive lighting
- artwork-informed materials
- environmental continuity

The interface should remain structurally consistent while allowing each entertainment world to possess its own atmosphere.

Brand identity must remain recognisable.

Atmosphere must never replace branding.

Future Direction 05

Device Independence

The user's world should not depend upon the device they happen to be using.

Desktop.

Tablet.

Television.

Mobile.

Automotive.

Future devices.

Each should represent the same underlying world using an interaction model appropriate for the device.

The world remains constant.

Only its expression changes.

Future Direction 06

Companion Intelligence

The role of the companion should become progressively more helpful without becoming more intrusive.

Future companion capabilities may include:

- remembering unfinished experiences
- surfacing useful relationships
- anticipating contextual information
- simplifying discovery

The companion should never become conversational for its own sake.

Nor should it interrupt entertainment unnecessarily.

The companion exists to reduce effort.

Not increase interaction.

Future Direction 07

Extension Ecosystem

The Mosaic extension ecosystem should strengthen rather than fragment the product.

Future extension systems should prioritise:

- shared terminology
- shared interaction models
- shared composition rules
- shared visual language

Extensions should feel native.

Users should rarely distinguish between:

Core Mosaic

and

Community extensions.

Future Direction 08

Accessibility As A Design Driver

Accessibility should continue evolving from:

> *compliance*

towards

> *experience quality.*

Future accessibility work should consider:

- cognitive accessibility
- reduced motion
- adaptive density
- alternative composition strategies
- assistive interaction models

Accessibility should improve every experience rather than existing as a separate mode.

Things We Expect To Learn

The following assumptions should be continuously validated.

- Does adaptive composition genuinely reduce cognitive effort?
- Does contextual assistance increase trust?
- Does the companion metaphor remain effective as Mosaic grows?
- Which interaction models become intuitive over time?
- Which concepts require simplification?

Design language should evolve through evidence rather than fashion.

Things We Expect To Reject

The following directions are unlikely to align with MDL unless compelling evidence emerges.

- engagement optimisation
- infinite promotional feeds
- algorithmic persuasion
- interface-first design
- attention-maximising notifications
- arbitrary visual novelty

Future contributors proposing these approaches should provide evidence that they strengthen, rather than weaken, the vision established by MDL-001.

Long-Term Vision

The ultimate ambition of Mosaic is not to become invisible through minimalism.

It becomes invisible because users trust it.

When trust exists:

People stop thinking about software.

They simply continue enjoying entertainment.

That remains the long-term objective regardless of how the underlying platform evolves.

Evolution Model

```
[mermaid]
flowchart LR
    Vision --> Principles
    Principles --> Systems
    Systems --> Features
    Features --> Learning
    Learning --> RefinedSystems["Refined Systems"]
    RefinedSystems --> BetterExperience["Better Experience"]
```

Notice that the vision never changes.

Only the understanding of how best to realise it.

Architectural Decisions

ADR	Decision
ADR-037	Future implementation should evolve without redefining the core philosophy.
ADR-038	Information is expected to become increasingly independent from presentation.
ADR-039	Accessibility is considered a driver of better design rather than a compliance exercise.
ADR-040	Future innovation must strengthen the companion philosophy rather than replace it.

Review Status

Status

Draft

Outstanding Questions

The Information-Driven Experience model should be formalised during MDL-003 (Mental Model) and MDS-003 (Composition Engine).

Next File

`glossary.md`

Glossary

Purpose

This glossary defines terminology specific to the Mosaic Design Language.

Many of these terms deliberately differ from traditional UI or design system vocabulary.

Future specifications should reference these definitions rather than redefining terminology.

Where a term is defined here, that definition takes precedence throughout the MDL and MDS.

Maintaining a controlled glossary helps ensure consistent communication between designers, engineers and contributors.
[oai_citation:0‡W3C](https://www.w3.org/WAI/WCAG22/Techniques/general/G62?utm_source=chatgpt.com)

A

Adaptive

The ability for an experience to reorganise itself in response to user context while preserving understanding.

Adaptive does **not** mean unpredictable.

Atmosphere

The emotional tone of a composition.

Atmosphere is influenced primarily by entertainment artwork while remaining constrained by the Mosaic Material System.

Atmosphere never replaces brand identity.

C

Companion

The primary behavioural metaphor of Mosaic.

A companion quietly assists the user, provides useful context and then steps aside.

Mosaic should never behave like a salesperson, dashboard or recommendation engine.

Composition

The current arrangement of information presented around a user's focus.

Unlike a page, a composition evolves over time.

Compositions reorganise.

Pages navigate.

Context

The user's current activity.

Examples include:

- watching an episode
- reading a novel
- listening to an album

Context determines relevance.

Context never permanently restricts exploration.

D

Domain

A major category of entertainment.

Examples include:

- Television
- Anime
- Books
- Films
- Music

Domains may contain multiple compositions.

Moving within a domain should generally feel less disruptive than moving between domains.

E

Enhancement

Providing additional value that deepens an existing experience.

Examples include:

- next episode information
- soundtrack
- source material
- production notes

Enhancement differs from persuasion.

F

Focus

The entertainment currently deserving primary attention.

Every composition has one current focus.

Focus changes naturally over time.

Changing focus should cause the composition to evolve.

I

Immersion

The reduction of mental effort required to remain inside an entertainment experience.

Immersion is the primary optimisation target of the Mosaic Design Language.

Information

A discrete piece of knowledge about the user's entertainment.

Examples include:

- watch progress
- chapter progress
- release date
- rating
- cast member

Future specifications may treat information as the primary architectural unit from which interfaces are composed.

M

Material

A visual surface used by Mosaic to communicate hierarchy.

Examples are defined within the Mosaic Design System.

Materials communicate structure.

Artwork communicates emotion.

Mental Model

The conceptual understanding users develop about how Mosaic works.

The Mosaic mental model intentionally centres around:

World

↓

Focus

↓

Context

↓

Composition

rather than pages and navigation.

P

Persuasion

Attempting to redirect attention towards different entertainment.

Persuasion is intentionally outside the scope of the core Mosaic experience.

Principle

A decision-making rule.

Principles are derived from the philosophy established within MDL-001.

They help contributors choose between competing solutions.

R

Relationship

A meaningful connection between pieces of entertainment or information.

Examples include:

- adaptation
- sequel
- soundtrack
- author
- cast
- franchise

Relationships provide context.

Future specifications are expected to formalise relationship modelling further.

T

Tile

The visual representation of information within a composition.

Tiles are not the information itself.

They are one possible expression of that information.

Tiles may adapt in:

- size
- emphasis
- placement

while preserving their identity.

Trusted Companion

The intended personality of Mosaic.

Characteristics include:

- knowledgeable
- calm
- friendly
- sophisticated
- respectful

This metaphor should guide future design decisions.

W

World

The user's complete entertainment universe.

A world consists of:

- collections
- history
- interests
- relationships

- current focus

The interface should communicate the user's world rather than isolated libraries.

Cross References

Specification	Primary Terms
MDL-002 Principles	Context, Companion, Immersion
MDL-003 Mental Model	World, Focus, Composition
MDL-004 Interaction Model	Composition, Tile, Domain
MDL-005 Composition Model	Tile, Material, Relationship
MDS Specifications	Material, Atmosphere, Adaptive

Glossary Maintenance

This glossary is expected to evolve throughout the lifetime of Mosaic.

New terms should only be introduced when:

- existing terminology cannot accurately describe a concept
- the concept is expected to appear across multiple specifications
- the definition improves consistency

Terminology should remain stable once established.

Renaming core concepts should require a formal design review.

Review Status

Status

Draft

Next File

`references.md`

References

Purpose

This document records the external references that informed MDL-001.

MDL is intentionally **not** derived from any single design system.

Instead, Mosaic synthesises ideas from multiple disciplines including:

- Human-centred design
- Product design
- Architecture
- Systems thinking
- Information architecture
- Software architecture

These references exist to provide context for future contributors.

They are **not** normative requirements.

Reading Order

Future contributors are encouraged to read references in the following order.

1. Product Vision 2. Human Interface Design 3. Design Systems 4. Architecture Decision Records 5. Systems Architecture 6. Documentation Standards

Human Interface Design

Apple Human Interface Guidelines

Apple Inc.

https://developer.apple.com/design/human-interface-guidelines/

Referenced throughout MDL for:

- experience-first thinking
- consistency
- hierarchy
- interaction philosophy

Not adopted directly.

Material Design

Google

https://m3.material.io/

Referenced for:

- design system organisation
- documentation structure
- token hierarchy

Material Design intentionally differs philosophically from MDL.

Fluent Design

Microsoft

https://fluent2.microsoft.design/

Referenced for:

- adaptive interface systems
- cross-device thinking
- design governance

Architecture

Architectural Decision Records

Michael Nygard

https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Referenced for:

- ADR structure
- architectural traceability
- decision preservation

Additional ADR guidance:

- Martin Fowler — Architecture Decision Record
- UK Government ADR Framework
- AWS ADR Guidance [oai_citation:0‡martinfowler.com](https://martinfowler.com/bliki/ArchitectureDecisionRecord.html?utm_source=chatgpt.com)

RFC Process

RFC Editor

https://www.rfc-editor.org/

Referenced for:

- document lifecycle
- versioning
- governance
- specification process

Relevant examples:

- RFC 2026
- RFC 3426
- RFC 7764 [oai_citation:1‡RFC Editor](https://www.rfc-editor.org/rfc/rfc2026.html?utm_source=chatgpt.com)

Documentation

Markdown Documentation

Markdown is the canonical source format for MDL.

Generated formats including:

- PDF
- HTML
- mdBook
- MkDocs

are considered publication artefacts.

The Markdown repository remains authoritative.

Documentation As Code

Referenced principles include:

- version control
- peer review
- pull requests
- changelog
- traceability

Documentation should evolve using the same workflow as software.

Product Philosophy

The following product philosophies strongly influenced MDL.

Experience Before Interface

Software should support experiences rather than become experiences.

Systems Before Features

Long-term maintainability is achieved through reusable systems rather than isolated feature development.

Calm Technology

Technology should remain in the background until needed.

While not directly implemented, this philosophy aligns closely with the intended behaviour of Mosaic as a trusted entertainment companion.

Design Language Inspirations

MDL intentionally studies—but does not attempt to replicate—the following systems.

System	Inspiration
Apple Human Interface Guidelines	Product philosophy
Material Design	Documentation structure
Fluent Design	Cross-device thinking
Carbon Design System	Enterprise governance
Atlassian Design System	Documentation quality
IBM Enterprise Design Thinking	Decision making

MDL deliberately avoids inheriting visual identity from any existing design language.

Its objective is to create an original philosophy suitable for entertainment software.

Mosaic-Specific Sources

The following documents directly informed MDL-001.

Founder Discovery Workshops

Conducted during the initial design engagement.

Topics included:

- Product Vision
- Product Beliefs

- Mental Model
- Companion Philosophy
- Adaptive Composition
- Information Architecture
- Material Metaphor

These workshops are considered the primary source for MDL.

Mosaic Platform Roadmap

The Mosaic roadmap establishes the long-term technical direction of the platform.

MDL intentionally remains implementation independent while ensuring philosophical alignment with that roadmap.

Normative References

The following documents should be considered required reading for contributors working on MDL.

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model

Informative References

The following documents provide additional context.

- MDS-001 Design Token Architecture
- MDS-002 Material System
- MDS-003 Composition Engine
- Future ADRs
- Engineering RFCs

These documents may evolve independently while remaining consistent with MDL.

Living References

This reference list is intentionally expected to evolve.

New references should only be added when they:

- influenced a significant design decision,

- provide historical context,
- explain architectural reasoning, or
- materially improve contributor understanding.

References should not become an exhaustive catalogue of design literature.

Quality is preferred over quantity.